GEOATLAS

A critical engineering review of the six most underestimated areas in the GeoAtlas architecture — with concrete, production-ready solutions.

	Engineering Review
	GeoAtlas Intelligence Platform
	1.0
	March 2026
	Internal / Confidential

Table of Contents

SECTION 01

Executive Summary

Six critical areas requiring production-grade redesign

The GeoAtlas architecture documents present a coherent high-level vision. However, a production readiness review identifies six areas where the current design is significantly underspecified, underestimated, or carries architectural risk that would cause system failure or quality degradation at scale. This report addresses each area with concrete, actionable production-grade solutions.

Area	Severity	Current State	Impact if Unresolved
NLP Pipeline	CRITICAL	4-week estimate for classifier	Wrong event classification → bad predictions
Prediction Engine	CRITICAL	No training data strategy	Models with no ground truth to train on
Asset Mapping	HIGH	Simple if/else rules	Misses nuanced event-asset relationships
Microservices	HIGH	6 services from day one	Massive operational overhead for MVP
Market Data Costs	MEDIUM	Free tiers assumed	Rate limits block real-time feature
Data Quality	MEDIUM	No feedback loop defined	Dataset degrades silently over time

SECTION 02

NLP Pipeline

The most underestimated component in the entire system

Root Cause Analysis

The architecture documents allocate one week for NER extraction and one week for event classification. In practice, building a reliable geopolitical NLP classifier requires a labelled training corpus, domain adaptation of base models, evaluation loops, and human verification before any data is trusted. The realistic timeline is 6–10 weeks for a production-quality pipeline.

■ **Critical Gap:** The documents assume off-the-shelf spaCy NER will work on geopolitical text. Standard en_core_web_trf is trained on news and web text but does not reliably identify sanctions, trade policy actors, or supply chain entities without fine-tuning.

Production NLP Architecture

A production-grade NLP pipeline requires five distinct stages, each with its own model, evaluation metric, and failure mode:

Stage	Model	Input	Output	Accuracy Target
Language Detection	langdetect / fastText	Raw article	Language code	> 99%
Relevance Filter	DistilBERT classifier	Article text	Geopolitical: Y/N	> 92%
NER Extraction	spaCy + fine-tuned trf	Relevant articles	Entities + labels	> 88% F1
Event Classification	RoBERTa fine-tuned	Article + entities	Event type	> 85% F1
Sentiment Scoring	FinBERT	Article text	Sentiment score	Pearson > 0.75

Training Data Strategy

The single most important missing piece. A classifier is only as good as its training data. The following sources provide labelled geopolitical event data without requiring manual annotation from scratch:

- **GDELT Event Database** — 300M+ coded geopolitical events since 1979 with CAMEO codes. Free. Use as weak supervision labels.
- **ACLED (Armed Conflict Location & Event Data)** — 1M+ conflict events with structured labels. Free for researchers.
- **ICEWS (Integrated Crisis Early Warning System)** — US govt-funded event dataset with actor/action/target coding.
- **Reuters/AP historical archives** — Pair with GDELT dates to create text→event label pairs.
- **Human annotation (Phase 2)** — Once MVP is live, use internal tools to correct and relabel predictions.

Production Code Pattern — Confidence Gating

Never write an event to the database without a confidence gate. Low-confidence events should go to a human review queue, not production:

event_classifier.py

```
class EventClassifier:
CONFIDENCE_THRESHOLD = 0.72
HUMAN_REVIEW_THRESHOLD = 0.55

def classify(self, article: str) -> ClassificationResult:
result = self.model.predict(article)
if result.confidence >= self.CONFIDENCE_THRESHOLD:
return result.with_status('AUTO_APPROVED')
elif result.confidence >= self.HUMAN_REVIEW_THRESHOLD:
self.review_queue.enqueue(article, result)
return result.with_status('PENDING_REVIEW')
else:
return result.with_status('REJECTED')
```

Revised NLP Timeline

Week	Deliverable	Validation Gate
1–2	Data pipeline: GDELT ingestion + article storage	10k articles stored, deduplicated
3–4	Relevance classifier: DistilBERT on geo keywords	Precision > 90% on test set
5–6	NER fine-tuning on GDELT-paired text	F1 > 85% on held-out articles
7–8	Event type classifier (RoBERTa)	F1 > 82% across 7 event categories
9–10	FinBERT sentiment + confidence gating + review queue	< 5% false event rate
11–12	Integration testing + shadow mode vs. production	Latency < 30s per article

SECTION 03

Prediction Engine

The hardest component — currently underspecified

The documents list LSTM + Transformer + Gradient Boosting as the prediction models, but do not address the fundamental question: what data are these models trained on? Without a labelled historical dataset of event→market reactions, no model can be trained. This section defines the complete production-grade prediction architecture.

The Training Data Problem

■ **Critical Gap:** The prediction documents assume training data exists. It does not. Building it is a 2–3 month effort that must be treated as a separate workstream.

The ground truth dataset must be constructed by joining historical events with subsequent price movements. The construction pipeline:

- **Step 1 — Event Corpus:** Use GDELT + ACLED to get 50,000+ historical geopolitical events with timestamps.
- **Step 2 — Price Alignment:** For each event, fetch T+1h, T+6h, T+24h, T+7d price data for affected assets via Polygon historical API.
- **Step 3 — Label Generation:** Compute percentage change. Apply thresholds: >2% = positive signal, <-2% = negative, else = neutral.
- **Step 4 — Feature Engineering:** Event type, sentiment score, region, sector, macro regime (VIX level, rate environment).
- **Step 5 — Train/Test Split:** Split chronologically, not randomly. Never let future events leak into training.

Production Model Architecture

A single model is insufficient. Production requires an ensemble with specialised sub-models, each targeting a specific prediction horizon:

Model	Type	Horizon	Input Features	Output
ShortPulse	FinBERT + Linear head	1h – 6h	Sentiment, event type, VIX	Direction + prob
TrendForce	Gradient Boosting (XGBoost)	24h – 7d	Macro indicators, sector, hist. reactions	% change range
VolatilityNet	LSTM	1h – 24h	Price history, options IV, volume	Volatility spike prob
RegimeFilter	Random Forest	Always-on	VIX, yield curve, DXY, sector flows	Market regime

Prediction Validation Framework

Every prediction must be tracked against actual market outcomes. This is not optional — it is the foundation of user trust and model improvement:

prediction_tracker.py

```
# predictions table — add these columns:
predicted_direction VARCHAR(8) -- 'UP' | 'DOWN' | 'NEUTRAL'
predicted_change_pct FLOAT
predicted_at TIMESTAMPTZ
resolve_at TIMESTAMPTZ -- T + horizon
actual_change_pct FLOAT -- filled by cron job at resolve_at
outcome VARCHAR(8) -- 'CORRECT' | 'WRONG' | 'PARTIAL'
model_version VARCHAR(16)

-- Accuracy dashboard query:
SELECT model_version,
COUNT(*) FILTER (WHERE outcome='CORRECT') * 100.0 / COUNT(*) AS accuracy,
AVG(ABS(predicted_change_pct - actual_change_pct)) AS avg_error
FROM predictions
GROUP BY model_version, prediction_horizon;
```

✓ **Production Rule:** Ship v1 predictions only for event types where back-test accuracy exceeds 60%. Display accuracy score on the UI. Transparency builds more trust than hiding uncertainty.

SECTION 04

Asset Mapping

Rule-based logic will fail at production scale

The current architecture maps events to assets using static sector rules (if sector == 'Energy': assets = ['XOM', 'CVX', 'BP']). This approach breaks immediately when events cross sectors, when new assets enter the market, or when an event has nuanced, company-specific implications. A knowledge graph is the correct production solution.

■ **High Risk:** A "China rare earth restriction" event is not just a Semiconductors event. It affects EV manufacturers, defence contractors, wind turbine companies, and specific mining stocks — none of which appear in a flat sector→ticker list.

Knowledge Graph Design

Replace the flat sector→asset list with a multi-dimensional relationship graph stored in PostgreSQL (using recursive CTEs) or Neo4j for complex traversals:

knowledge_graph_schema.sql

```
-- Nodes
CREATE TABLE kg_entities (
id UUID PRIMARY KEY,
entity_type VARCHAR(20), -- 'ASSET' | 'SECTOR' | 'COUNTRY' | 'COMMODITY'
name VARCHAR(100),
metadata JSONB
);

-- Edges (directional relationships)
CREATE TABLE kg_relationships (
id UUID PRIMARY KEY,
source_id UUID REFERENCES kg_entities(id),
target_id UUID REFERENCES kg_entities(id),
relationship VARCHAR(40), -- 'SUPPLIES_TO' | 'COMPETES_WITH'
strength FLOAT, -- 0.0 to 1.0
data_source VARCHAR(30), -- 'MANUAL' | 'SEC_FILING' | 'ML_DERIVED'
last_verified DATE
);
```

Dynamic Mapping Layers

Asset mapping should operate in three layers, applied in order of specificity:

Layer	Method	Coverage	Latency
L1 — Direct Mention	NER entity matching to asset ticker	~40% of events	< 10ms

L2 — Supply Chain	Knowledge graph traversal (2-hop)	~35% of events	< 50ms
L3 — Sector Expansion	Sector→asset with relevance weighting	~20% of events	< 20ms
L4 — ML Similarity	Embedding similarity on past events	~5% (edge cases)	< 200ms

Seeding the Knowledge Graph

The graph does not need to be built manually from scratch. These free data sources can seed 80%+ of the required relationships:

- **SEC EDGAR filings** — 10-K filings list key suppliers and customers. Parse with EDGAR full-text search API (free).
- **OpenCorporates** — Company-country-sector relationships. Free tier available.
- **Wikidata SPARQL** — Industry classifications, parent companies, subsidiaries. Completely free.
- **FactSet/Refinitiv supply chain data** — Paid, but consider for institutional tier clients only.
- **ML-derived edges** — Train a co-movement model: if two assets consistently move together after events, add a 'CORRELATED' edge.

SECTION 05

Microservices Architecture

Six services from day one is an operational anti-pattern for an MVP

The architecture documents specify six independent microservices at launch: Auth, User, Market, Event Intelligence, Prediction, and Board. Each service requires its own deployment pipeline, health checks, inter-service networking, distributed tracing, and on-call runbook. For a team building an MVP, this overhead consumes engineering time that should go toward the product.

■ **High Risk:** Martin Fowler's "Microservice Premium" principle states: microservices add complexity that only pays off at team scale (5+ engineers per service). Premature decomposition is one of the most common reasons early-stage products fail to ship.

The Modular Monolith Approach

A modular monolith gives you the internal code organisation of microservices without the operational overhead. Each module is a Python package with strict import boundaries — it can be extracted into a service later with minimal refactoring:

project structure — modular monolith

```
geoatlas/
■■■ main.py # FastAPI app, mounts all routers
■■■ core/
■ ■■■ config.py
■ ■■■ database.py
■■■ modules/
■ ■■■ users/ # User module — owns its own models + routes
■ ■ ■■■ router.py
■ ■ ■■■ service.py
■ ■ ■■■ models.py
■ ■■■ events/ # Event intelligence module
■ ■ ■■■ router.py
■ ■ ■■■ nlp_pipeline.py
■ ■ ■■■ models.py
■ ■■■ market/ # Market data module
■ ■■■ predictions/ # Prediction module
■ ■■■ boards/ # Boards module
■■■ workers/ # Celery tasks (can become separate worker later)
```

Migration Strategy — When to Split

Extract a module into a standalone service only when one of these thresholds is met:

Trigger	Module to Extract	Reason
NLP pipeline consuming > 60% of CPU	Event Intelligence	GPU inference needs dedicated resources
ML predictions taking > 5s	Prediction Service	Needs GPU, separate scaling
Market data cache invalidating too frequently	Market Service	High-frequency vs. low-frequency mismatch
User auth becoming a security audit target	Auth Service	Compliance isolation
Team size reaches 6+ engineers	All modules	Conway's Law — structure follows org

✓ **Recommendation:** Start with a single FastAPI monolith + Celery workers. Extract the NLP pipeline and Prediction Service as standalone workers in Month 3, when their resource profiles diverge from the main application.

SECTION 06

Market Data API Cost Architecture

Free tiers cannot support a real-time platform

The product requires real-time prices for stocks, ETFs, commodities, crypto, and forex across hundreds of assets. The architecture documents suggest starting with free API tiers, but a realistic analysis of those limits shows they are incompatible with the stated feature set.

API Tier Reality Check

Provider	Free Tier Limit	Adequate for MVP?	Paid Tier Cost
Polygon.io	5 calls/min, delayed 15min	No — delayed data breaks real-time	$29/mo starter
Finnhub	60 calls/min, delayed	No — no WebSocket on free	$50/mo basic
Alpha Vantage	5 calls/min, 500/day	No — 500/day < 1 asset real-time	$50/mo
Twelve Data	8 calls/min, 800/day	Marginal — only for < 10 assets	$29/mo
Yahoo Finance	Unofficial, unreliable	No — no SLA, breaks regularly	N/A

Production Caching Architecture

The correct strategy is not to upgrade API tiers immediately, but to cache aggressively and fetch intelligently. With proper caching, a $29/mo Polygon plan can serve hundreds of concurrent users:

market_cache_strategy.py

```
CACHE_POLICY = {
'realtime_price': {'ttl': 15, 'layer': 'redis'},
'daily_ohlcv': {'ttl': 3600, 'layer': 'redis'},
'historical_1y': {'ttl': 86400, 'layer': 'postgres'},
'fundamentals': {'ttl': 604800,'layer': 'postgres'},
}
```

```
# WebSocket multiplexing: ONE connection to Polygon,
# fan-out to ALL subscribed users via internal pub/sub
class MarketDataHub:
def __init__(self):
self.polygon_ws = PolygonWebSocket(api_key=...)
self.subscribers: dict[str, list[WebSocket]] = {}
```

```
async def broadcast(self, ticker, price_data):
for ws in self.subscribers.get(ticker, []):
await ws.send_json(price_data)
```

Recommended Budget Model (MVP Phase)

Item	Provider	Cost/Month	Covers
Real-time stock/ETF	Polygon Starter	$29	WebSocket, unlimited symbols
Crypto prices	CoinGecko Pro	$129	500+ coins, 30-day history
News API	NewsAPI Business	$449	500 req/day + archives
Forex/Commodities	Twelve Data	$29	Forex + commodity prices
Infrastructure (AWS)	EC2+RDS+Redis	~$120	2 EC2 t3.medium, RDS, ElastiCache
Total MVP		~$756	Sufficient for 0–5k users

SECTION 07

Data Quality & Feedback Loops

Without feedback, your proprietary dataset degrades silently

The documents repeatedly claim that the Event→Asset Impact dataset is the core competitive moat. This is only true if the data is accurate. A dataset of 10,000 poorly classified events is not a moat — it is a liability. Data quality must be a first-class engineering concern from day one, not a post-MVP cleanup task.

The Three-Layer Quality Pipeline

Layer 1 — Automated Quality Gates (runs before every database write):

- Confidence threshold gate: reject events with confidence < 0.55
- Schema validation: all required fields must be non-null
- Duplicate detection: SHA256 hash of (title + event_type + country + published_date)
- Temporal consistency: event timestamp must be within 48h of article publish date
- Asset validation: all tickers must exist in the assets table

Layer 2 — Human Review Queue (events in the 0.55–0.72 confidence band):

- Build a simple internal annotation UI — 10 minutes of engineering, not a full product
- Reviewer sees: article headline, extracted event, entity list, suggested impact
- Actions: Approve / Reject / Edit (inline correction of event_type, assets, impact)
- Approved corrections feed back into the training dataset automatically
- Target: 50–100 human-reviewed events per week in early months

Layer 3 — Outcome Verification (runs T+24h and T+7d after each event):

outcome_verifier.py (Celery task)

```
@app.task
def verify_event_outcome(event_id: str, horizon: str):
event = Event.get(event_id)
impacts = EventImpact.filter(event_id=event_id)

for impact in impacts:
price_at_event = get_price(impact.asset_id, event.published_at)
price_at_horizon = get_price(impact.asset_id,
event.published_at + timedelta(hours=parse(horizon)))
actual_change = (price_at_horizon - price_at_event) / price_at_event

# Update the record with ground truth
impact.update(
actual_change_pct=actual_change,
verified_at=datetime.utcnow()
)
```

Key Quality Metrics Dashboard

These metrics should appear in the internal Grafana dashboard from day one:

Metric	Target	Alert Threshold
Event classification accuracy	> 85%	< 75% → page on-call
NLP pipeline latency (p95)	< 30s	> 60s → alert
Events auto-approved rate	> 70%	< 50% → model retraining needed
Human review queue backlog	< 100	> 500 → increase reviewer bandwidth
Prediction accuracy (24h)	> 58%	< 50% → disable prediction feature
Asset mapping coverage	> 90%	< 80% → expand knowledge graph
News ingestion freshness	< 10 min lag	> 30min → check API health

SECTION 08

Revised Development Timeline

A realistic 6-month roadmap incorporating all improvements

The original documents propose a 4-month timeline. Incorporating the production-grade improvements above, a realistic timeline is 6 months to a quality MVP — one that can be shown to investors and early users without embarrassing prediction errors or pipeline failures.

Month 1 Foundation

- Modular monolith setup (FastAPI + PostgreSQL + Redis + Celery)
- News ingestion from 5 sources with deduplication
- Raw article storage + Kafka pipeline skeleton
- Knowledge graph schema seeded from Wikidata + SEC EDGAR
- Basic Next.js frontend — news feed only

Month 2 NLP Core

- Relevance classifier (DistilBERT) — geo/macro filter
- NER fine-tuning on GDELT-paired text
- Event type classifier (RoBERTa) — 7 categories
- Confidence gating + human review queue
- Internal annotation UI for review queue

Month 3 Intelligence Layer

- FinBERT sentiment integration
- 4-layer asset mapping system (direct → supply chain → sector → ML)
- event_impacts table population from NLP output
- Market Data Service with WebSocket multiplexing
- Event feed + market panel UI

Month 4 Prediction Engine

- Historical training dataset construction (GDELT + Polygon historical)
- ShortPulse model (1–6h) + TrendForce model (24h–7d)
- Prediction tracking + outcome verification pipeline

- Prediction UI with accuracy transparency display
- Data quality metrics dashboard (Grafana)

Month 5	Platform Features

- Pinterest-style boards + pins
- Personalized alerts (email + push)
- Global map visualization
- User watchlists
- Pro tier paywall (Stripe integration)

Month 6	Hardening & Mobile

- React Native mobile app (iOS + Android)
- Performance testing + load testing (k6)
- NLP service extraction from monolith (if CPU threshold hit)
- Security audit + penetration testing
- Public beta launch

SECTION 09

Production Risk Matrix

Consolidated risks with mitigations across all six areas

Risk	Probability	Impact	Mitigation	Owner
NLP classifier ships with < 75% accuracy	HIGH	CRITICAL	Confidence gating; human review queue; don't launch without 85%+ on test set	ML Lead
No training data for prediction models	CERTAIN	CRITICAL	GDELT + Polygon historical API pipeline must start in Month 1 as parallel workstream	Data Lead
Asset mapping misses supply chain events	HIGH	HIGH	Knowledge graph from Wikidata/SEC; 4-layer fallback system	Backend Lead
Microservice overhead delays MVP	HIGH	HIGH	Modular monolith first; split only when resource profiles diverge	Architect
Market data API costs exceed budget	MEDIUM	HIGH	WebSocket multiplexing; Redis cache with TTL policy; budget $756/mo from start	Infra Lead
Prediction accuracy falls below 50%	MEDIUM	CRITICAL	Track all predictions vs. outcomes; auto-disable feature if accuracy < 50%	ML Lead
Data quality degrades silently	HIGH	HIGH	Automated quality gates + Grafana metrics dashboard from Month 1	Data Lead
News source bias distorts event classification	MEDIUM	MEDIUM	5+ news sources; source diversity score per event; flag single-source events	Data Lead

Final Strategic Note

The GeoAtlas concept is compelling and the architecture direction is broadly correct. The six areas addressed in this report are not reasons to abandon the project — they are reasons to plan more carefully before building. The most important immediate actions are: (1) begin the historical training data pipeline in parallel with the main build from day one, (2) commit to the modular monolith pattern until at least Month 4, and (3) instrument quality metrics before shipping any NLP output to users. A product that does fewer things accurately is worth more than one that does many things unreliably.